

Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Лабораторная работа №1

по дисциплине «Информационная безопасность»

Разработка защищённого REST API с интеграцией в CI/CD

Выполнил:
Карпов Александр Дмитриевич
Р3413

Санкт-Петербург, 2026

Содержание

1	Цель работы	2
2	Постановка задачи	2
3	Используемые инструменты	2
4	Ход выполнения	2
4.1	Архитектура приложения	2
4.2	Защита от SQL-инъекций	3
4.3	Защита от XSS	3
4.4	Аутентификация	3
4.5	CI/CD	3
5	Результаты	3
5.1	Работа API	3
5.2	Отчёты SAST/SCA	5
6	Ссылки	7
7	Вывод	7
8	Ответы на контрольные вопросы	7

1 Цель работы

Получить практический опыт разработки безопасного backend-приложения с автоматизированной проверкой кода на уязвимости. Освоить базовые меры защиты из OWASP Top 10 (инъекции, XSS, недостатки аутентификации) и интеграцию инструментов анализа безопасности в CI/CD.

2 Постановка задачи

Разработать REST API со следующими эндпоинтами:

- `POST /auth/login` — аутентификация пользователя;
- `GET /api/data` — получение данных, доступно только аутентифицированным пользователям;
- `POST /api/buy` — собственный метод: покупка кристаллов (дополнительно реализован `POST /auth/register`).

Реализовать защиту от SQL-инъекций, XSS и небезопасной аутентификации; настроить CI/CD pipeline с запуском SAST- и SCA-сканеров.

3 Используемые инструменты

- Python 3.12, Flask, Flask-SQLAlchemy, SQLite;
- PyJWT — выпуск и проверка JWT;
- bcrypt — хэширование паролей;
- Bandit — статический анализ (SAST);
- pip-audit — анализ зависимостей (SCA);
- GitHub Actions — CI/CD;
- pytest — автотесты; curl — ручное тестирование.

4 Ход выполнения

4.1 Архитектура приложения

Приложение собрано по схеме application factory. Логика вынесена в пакет `app`: модели (`models.py`), выпуск и проверка токена (`auth.py`), маршруты (`routes.py`), начальное наполнение БД (`seed.py`). Точка входа `run.py` лишь создаёт приложение.

4.2 Защита от SQL-инъекций

Все обращения к БД выполняются через ORM SQLAlchemy (`filter_by`, `db.session.get`). Пользовательский ввод всегда передаётся как параметр запроса и экранируется драйвером, а не склеивается в строку SQL. Попытка входа с пейлоадом `admin' OR '1'='1` приводит к ответу 401 — инъекция не срабатывает.

4.3 Защита от XSS

Все строковые поля, пришедшие от пользователя (`username`, `color`), экранируются функцией `markupsafe.escape` перед возвратом в JSON. Имя вида `<script>alert(1)</script>` возвращается как `<script>`... и не исполняется на клиенте.

4.4 Аутентификация

Пароли хранятся только в виде `bcrypt`-хэша со случайной солью (`bcrypt.hashpw`). При успешном входе выдаётся подписанный JWT (алгоритм HS256) с временем жизни. Защищённые эндпоинты закрыты декоратором `token_required`, который проверяет наличие заголовка `Authorization: Bearer`, подпись токена и срок действия (`exp`).

4.5 CI/CD

Файл `.github/workflows/ci.yml` запускается на каждый `push` и `pull_request` и последовательно выполняет: установку зависимостей, Bandit (SAST), `pip-audit` (SCA) и `pytest`.

5 Результаты

5.1 Работа API

Логин и получение токена.

```
Terminal - login

$ curl -s -X POST localhost:5000/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"CorrectHorse42!"}'

{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwidXNlcm5hbWUiOiJhZG1pb2IiImplhdCI6MTc5MTM5MTI1OjCwiZlwIjoxNzIxMzk0ODU4fQ.bnXVwXpHQU-n6zXL_ZnLdjobInhnP2TGgfPcqSZMA9Y"}

```

Доступ к /api/data без токена (401) и с токеном (200).

```
Terminal - /api/data (401 vs 200)

$ curl -i -s localhost:5000/api/data # без токена
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.14.7
Date: Wed, 07 Oct 2026 16:40:59 GMT
Content-Type: application/json

$ curl -s localhost:5000/api/data \
  -H "Authorization: Bearer $TOKEN" # с токеном
{"balance":5000,"crystals":[{"color":"red","id":1,"price":100,"stock":50},{"color":"green","id":2,"price":120,"stock":40},{"color":"blue","id":3,"price":150,"stock":30},{"color":"violet","id":4,"price":300,"stock":10}], "user":"admin"}
```

SQL-инъекция в логине отклонена.

```
Terminal - SQL injection rejected

$ curl -i -s -X POST localhost:5000/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin\"' OR \"'1\"'='\"'1\"','password\":\"x\"}'
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.14.7
Date: Wed, 07 Oct 2026 16:40:59 GMT
Content-Type: application/json
Content-Length: 32
Connection: close

{"error":"invalid credentials"}
```

XSS: имя пользователя экранировано в ответе.

```
Terminal - XSS escaped

$ curl -s -X POST localhost:5000/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"username":"<script>alert(1)</script>","password":"password123"}'
{"id":2,"username":"&lt;script&gt;alert(1)&lt;/script&gt;"}

$ curl -s localhost:5000/api/data -H "Authorization: Bearer $XSS_TOKEN"
{"balance":1000,"crystals":[{"color":"red","id":1,"price":100,"stock":50},{"color":"green","id":2,"price":120,"stock":40},{"color":"blue","id":3,"price":150,"stock":30},{"color":"violet","id":4,"price":300,"stock":10}], "user":"&lt;script&gt;alert(1)&lt;/script&gt;"}
```

Покупка кристаллов.

```
Terminal - buy

$ curl -s -X POST localhost:5000/api/buy \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"crystal_id":1,"quantity":2}'
{"balance":4800,"color":"red","purchase_id":1,"quantity":2,"total_price":200}
```

5.2 Отчёты SAST/SCA

Bandit (SAST) — критических проблем не найдено.

```
GitHub Actions - SAST: Bandit

$ bandit -r app

Test results:
  No issues identified.

Code scanned:
  Total lines of code: 195
  Total lines skipped (#nosec): 0
  Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
  Total issues (by severity):
    Undefined: 0
    Low: 0
    Medium: 0
    High: 0
  Total issues (by confidence):
    Undefined: 0
    Low: 0
    Medium: 0
    High: 0
  Files skipped (0):
```

pip-audit (SCA) — уязвимых зависимостей не найдено.

```
●●● GitHub Actions – SCA: pip-audit

$ pip-audit -r requirements.txt --desc

No known vulnerabilities found
```

Автотесты (pytest) в составе pipeline.

```
●●● GitHub Actions – pytest

$ pytest -q

..... [100%]
7 passed in 5.94s
```

Успешный запуск pipeline (все шаги зелёные).

```
●●● GitHub Actions – pipeline (success)

$ gh run view --job=112901415655 --repo Alexander-D-Karpov/ib-lab1

✓ master CI · 37653150323
Triggered via push about 7 minutes ago

✓ security-and-tests in 28s (ID 112901415655)
  ✓ Set up job
  ✓ Run actions/checkout@v4
  ✓ Run actions/setup-python@v5
  ✓ Install dependencies
  ✓ SAST - Bandit
  ✓ SCA - pip-audit
  ✓ Tests
  ✓ Post Run actions/setup-python@v5
  ✓ Post Run actions/checkout@v4
  ✓ Complete job

View this run on GitHub:
https://github.com/Alexander-D-Karpov/ib-lab1/actions/runs/37653150323
```

6 Ссылки

- Репозиторий: <https://github.com/Alexander-D-Karpov/ib-lab1>
- Последний успешный pipeline:
<https://github.com/Alexander-D-Karpov/ib-lab1/actions/runs/376531>

7 Вывод

Разработан защищённый REST API с аутентификацией по JWT и хэшированием паролей bcrypt. Реализована защита от SQL-инъекций (ORM с параметризацией) и XSS (экранирование вывода). Настроен CI/CD pipeline с автоматическим запуском SAST (Bandit) и SCA (pip-audit) на каждый push и pull request. Все проверки безопасности и автотесты проходят успешно.

8 Ответы на контрольные вопросы

1. Почему bcrypt предпочтительнее SHA-256 для хранения паролей?

SHA-256 — быстрая хэш-функция общего назначения, и эта скорость играет против защиты: на GPU перебираются миллиарды хэшей в секунду. bcrypt спроектирован специально для паролей: он намеренно медленный, имеет настраиваемый фактор сложности (work factor), который можно повышать с ростом мощности железа, и встроенную случайную соль для каждого пароля. Соль защищает от радужных таблиц и не даёт одинаковым паролям давать одинаковые хэши.

2. Разница между SAST и DAST.

SAST (статический анализ) исследует исходный код или байт-код без запуска приложения и находит потенциально опасные конструкции (небезопасные вызовы, хардкод секретов). DAST (динамический анализ) тестирует уже запущенное приложение «снаружи», посылая запросы и наблюдая реакцию, и находит уязвимости времени выполнения и конфигурации. SAST применяется раньше и видит код, но даёт ложные срабатывания; DAST видит реальное поведение, но не знает внутренностей.

3. Как работает JWT?

JWT состоит из трёх частей, разделённых точкой: header (алгоритм и тип), payload (claims — например **sub**, **exp**) и signature. Header и payload — это

base64url от JSON, они не шифруются, а только кодируются. Подпись считается по header и payload с секретным ключом сервера (для HS256 — HMAC-SHA256). При проверке сервер пересчитывает подпись по полученным header и payload и сравнивает с присланной: если ключ верный и данные не менялись — подписи совпадут. Дополнительно проверяется срок действия `exp`. Так сервер доверяет токену, не храня сессию.

4. Чем опасны непроверенные сторонние зависимости?

В зависимостях (и их транзитивных зависимостях) могут быть известные уязвимости, которые наследуются приложением. Атакующему достаточно использовать публично описанную CVE в устаревшей библиотеке. Без регулярного аудита (SCA) такие дыры остаются незамеченными; отдельные риски — supply-chain атаки и typosquatting (вредоносные пакеты с похожими именами). Поэтому зависимости фиксируют по версиям и автоматически сканируют в CI.